Accountable AI Inference Services via Smart Contracts

Vincenzo Botta[2], Ivan Visconti[1], Andrea Vitaletti[1], and Marco Zecchini[1]

[1] Sapienza University of Rome, Rome, Italy
{visconti,vitaletti,zecchini}@diag.uniroma1.it
[2] Independent researcher botta.vin@gmail.com

Abstract. There is nowadays a quickly growing market of AI-based services available to users upon subscription.
In general users get a free trial period in which they evaluate the quality of the AI-based service and then pay for an unrestricted use. However, 1) there is no guarantee that the server will maintain the same quality provided during the trial phase, and 2) there is no guarantee that the client will want to subscribe after having received during the trial period everything she needed.
In this work we study accountability of such AI inference services. We show the design and analysis of a construction in which a user with a confidential benchmark can appeal to a smart contract in case the quality of the service drops down (measuring this with the benchmark) after the subscription. We provide also some experimental results confirming the viability of our approach for improving the robustness of AI-based services via blockchain technology.

1 Introduction

There is nowadays a large use of remote machine learning models. Users send queries to a server that runs a trained model and returns the prediction. These systems are commonly referred as AI inference services.

AI inference services are increasingly deployed through subscription-based interfaces. A client wishing to use a proprietary model typically pays for continued API access, often after evaluating whether the model satisfies a desired quality performance. This seemingly simple interaction hides a subtle trust problem: before committing to payment, the client needs assurance of quality while, at the same time, the server does not want to give temporary free access as it might give already to the clients what they need, and thus they would not need anymore to pay for a subscription. We will now elaborate more in detail the main use case that we address in our work.

Issues in AI inference services. Consider a client who defines a benchmark consisting of queries and correct answers, with the purpose of evaluating the quality of some model.

A malicious client could send queries to the server and could then evaluate the model according to the benchmark checking the correctness of the answers. However, from the server's perspective, the benchmarking phase could correspond to providing an unpaid service. Indeed, the client might have received everything she needed already in that phase (i.e., a client might ask questions without knowing the answers, and after having received the desired answers she might just claim that the quality is below the private threshold, refusing to start a subscription to the service).

On the other hand, a malicious server could leverage a high-quality external model. During the benchmarking phase, the server could answer correctly through the use of this external model, while, after the subscription, the server could answer using a worse local model. In this way, the client is willing to subscribe to the service but is then damaged by the lower quality of the service after the payment. Notice that one can not simply get an evidence of the worse quality by repeating the same query, since the server can obviously record the answers and therefore it can replay the previous answers for consistency.

The above discussion illustrates a fundamental two-sided vulnerability: the client must not be able to get answers without paying, and the server must not be able to decrease the quality of the service after subscription.

Addressing both sides of this vulnerability requires a mechanism that binds the server to a consistent behavior. In contrast to previous approaches, based on model commitment and zero-knowledge proofs (discussed in Section 2), in this work, we show a smart contract design that enforces private pre-purchase evaluation and post-purchase accountability without proving every inference step.

Contribution of the paper. We introduce *Accountable Subscription for AI Inference (ASAI)*, a protocol that reveals no model parameters, architecture, circuit structure, or intermediate activations during benchmarking, only a single decision bit is disclosed prior to payment, indicating whether the benchmark is met. We construct a concrete protocol achieving benchmark and output privacy under standard cryptographic assumptions. By separating subscription governance from full proof-of-inference, our approach provides a lightweight and deployable cryptographic layer for AI service accountability without the prohibitive cost of complete neural-network zero-knowledge proofs.

1.1 Technical Overview

At a high level, our ASAI involves three actors: a *client* wishing to evaluate and potentially subscribe to an AI inference service, a *server* hosting a proprietary machine learning model $\mathcal{M}$ and offering access through a subscription, and a *smart contract* S deployed on a public blockchain that acts as a third party, storing commitments, holding deposits, and adjudicating disputes.

Our ASAI separates the subscription lifecycle into three logically distinct phases (Fig.1): *commitment and challenge reservation*, *private pre-purchase evaluation*, and *post-purchase accountability*. We now briefly describe these phases.

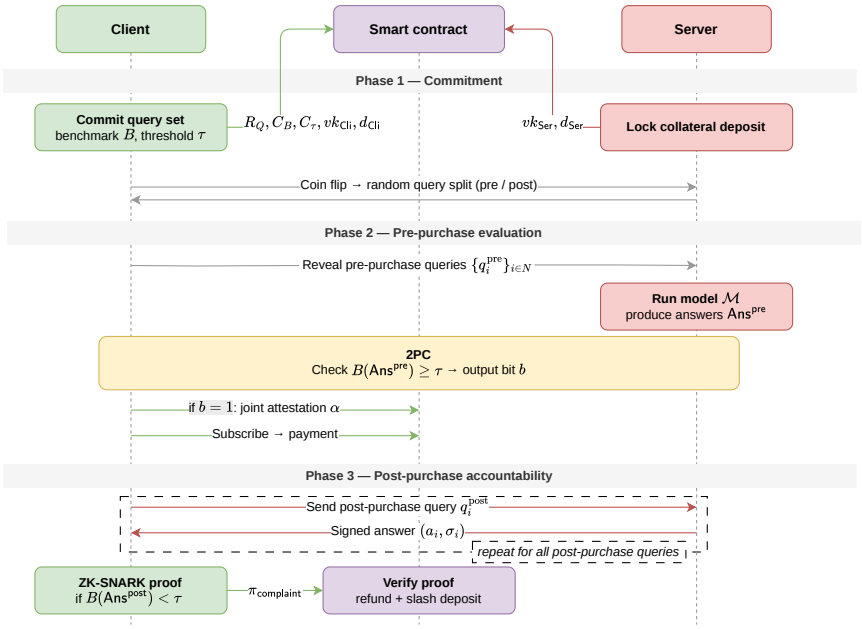

Fig. 1: Architecture of Accountable Subscription for AI Inference system.

Commitment and Challenge Reservation. In the first phase, the parties share and upload to the smart contract all the material that is necessary to guarantee protocol correctness in future phases.

The client prepares a large set of N *pairs* of queries (e.g., $N = 10000$). Concretely, it constructs N pairs $(q_1^{(0)}, q_1^{(1)})$, $(q_2^{(0)}, q_2^{(1)})$, $\ldots$, $(q_N^{(0)}, q_N^{(1)})$. For each $i \in [N]$, it creates two leaf values $v_i^{(0)} = q_i^{(0)} \,||\, 0$ and $v_i^{(1)} = q_i^{(1)} \,||\, 1$ and she builds a Merkle tree over these $2N$ leaf values $v_1^{(0)}, v_1^{(1)}, v_2^{(0)}, v_2^{(1)}, \ldots, v_N^{(0)}, v_N^{(1)}$. Finally, the client publishes the Merkle root R_Q to the smart contract S.

The client also publishes: (1) a commitment $C_B = \mathsf{Com}(B; r_B)$ to a benchmark function B, (2) a commitment $C_\tau = \mathsf{Com}(\tau; r_\tau)$ to the correctness threshold τ (fraction of correct answers required), (3) its public verification key $\mathsf{vk}_{\mathsf{Cli}}$, (4) and locks a deposit d_{Cli} into S. The server registers its public verification key $\mathsf{vk}_{\mathsf{Ser}}$ and locks a collateral deposit d_{Ser} into S.

Next, the client and server execute a *fair coin-tossing protocol* using fresh private randomness ρ_{Cli} and ρ_{Ser} to jointly generate a public random bit string $\mathbf{b} = (b_1, \ldots, b_N) \in \{0,1\}^N$. The hash of the resulting $\mathbf{b}$ is recorded on-chain. For each $i \in [N]$, bit b_i designates which query from pair i is used for pre-purchase evaluation (i.e., $q_i^{\mathrm{pre}} := q_i^{(b_i)}$, the complementary bit $1 - b_i$ reserves the post-purchase query $q_i^{\mathrm{post}} := q_i^{(1-b_i)}$).

In the pre-purchase evaluation, for each i, the client will reveal only $v_i^{(b_i)} = q_i^{(b_i)} \,||\, b_i$ together with its Merkle proof for the root R_Q and, in a post-purchase challenge, on index i, the client uses the *complementary* leaf $v_i^{(1-b_i)}$.

Private Pre-Purchase Evaluation. The parties evaluate the model $\mathcal{M}$ exclusively on the pre-purchase queries $\{q_i^{\text{pre}}\}_{i=1}^N$.

The client opens the corresponding Merkle leaves in the R_Q tree to the server, proving that each revealed q_i^{pre} belongs to the committed pair in the right position. The server evaluates $\mathsf{Ans}^{\text{pre}} = \{\mathcal{M}(q_i^{\text{pre}})\}_{i=1}^N$ and submits to S a signed commitment $\sigma_1 = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}\big(\mathsf{Com}(\mathsf{Ans}^{\text{pre}}; r_{\mathsf{Ans}})\big)$.

The parties then run a two-party computation (2PC) whose private inputs are for the client $(B, r_B, \tau, r_\tau, \mathsf{sk}_{\mathsf{Cli}})$ and for the server $(\mathsf{Ans}^{\text{pre}}, r_{\mathsf{Ans}}, \mathsf{sk}_{\mathsf{Ser}})$.

The 2PC computes the predicate $B(\{q_i^{\text{pre}}\}_{i=1}^N, \mathsf{Ans}^{\text{pre}}) \geq \tau$ and returns a bit b indicating the success or the failure of the check (i.e., whether the fraction of correct answers meets the threshold τ). If $b = 1$, the 2PC additionally produces a joint attestation $\alpha = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Cli}}}\Big(\mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}\big(b,\, R_Q,\, \mathbf{b}\big)\Big)$, which is then submitted to S by one of the party. The contract verifies both signatures. If $b = 0$, no attestation is produced.

Post-Purchase Accountability. After subscribing, if the client suspects model degradation, it may challenge the server using the complementary queries $\{q_i^{\text{post}}\}_{i \in N}$. The server returns each answer $a_i = \mathcal{M}(q_i^{\text{post}})$ together with a per-query signature $\sigma_i = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}(H(q_i^{\text{post}}),\, a_i)$.

If the client believes the model has degraded, it constructs a ZK-SNARK proof $\pi_{\mathsf{complaint}}$ certifying:

1. correct opening of C_B and C_τ (revealing the original B and τ),
2. correct opening of the queries used,
3. validity of all server signatures σ_i on the pair $(q_i^{\text{post}},\, a_i)$,
4. and, finally, that $B(\{q_i^{\text{post}}\}_{i \in S}, \{a_i\}_{i \in S}) < \tau$.

The client submits $\pi_{\mathsf{complaint}}$, the Merkle proofs, and commitments to S. If the verification passes, S issues a refund to the client and slashes (part of) d_{Ser}.

ASAI correctness. The Merkle tree with the bit tags for every leaf, the binding commitments and the ZK-SNARK together prevent the client from cheating (fabricating complaints or changing the benchmark), while the signatures on each query prevent the server from repudiating its answers. This composition guarantees that a subscription is only issued when the model passes the benchmark on a random half of the queries, and any statistically significant degradation can be reliably detected and punished afterwards. Indeed, suppose the server's model is weak, meaning that its expected fraction of correct answers on any set of queries is at most $1 - \varepsilon < \tau$ (we remind the reader that τ is fraction of correct answers desired by the client). Let $\varepsilon' = \tau - (1 - \varepsilon) > 0$ denote the *degradation gap*. By construction, the post-purchase queries are independent of the pre-purchase queries, so the event that the server passes the pre-purchase benchmark carries

no information about its post-purchase score. Applying the Chernoff bound[3] to the post-purchase queries, the probability that a weak server passes the post-purchase benchmark decreases exponentially in N and in ε'^2. Concretely, for $\varepsilon' = 0.1$ and $N = 5000$ query pairs, the escape probability is at most 2^{-128} (for more details see Sec. 6). For a larger degradation gap of $\varepsilon' = 0.3$, already $N = 2000$ query pairs suffice to achieve the same guarantee.

Moreover, notice that throughout the entire protocol, no model weights, architecture details, or intermediate activations of $\mathcal{M}$ are ever disclosed. The only information revealed to the client before payment is the output bit b of the 2PC. Full query answers become visible to the client only after subscription and in the event of a post-purchase complaint, to produce the ZK-SNARK proof.

2 Related Works

Zero-knowledge proofs for machine learning inference have received significant attention in recent years. Systems such as zkCNN [LXZ21], zkLLM [SLZ24], zkGPT [QSL+25], and the ZKML compiler [CWSK24] construct succinct proofs that a model was executed correctly on a given input. These works (and the broader line of research surveyed in [PWZ+25]) enable verification of model outputs while keeping weights private. Proofs-of-training [APKP24] extend the idea to the training phase. While these approaches provide strong per-inference verifiability, they require arithmetizing the entire network, resulting in proving costs that remain computationally heavy, with the state-of-the-art still requiring several minutes of GPU time per inference for large models [SLZ24, QSL+25].

Recent work has explored alternatives to full ZK proofs. *opML* [CSYW24] proposes an optimistic machine-learning framework on blockchain that uses interactive fraud proofs and a multi-phase dispute game to enable efficient on-chain verification of large models. Another line of research introduces *lightweight cryptographic proofs of inference* [ACC+26], in which the prover commits to the execution trace via Merkle-tree-based vector commitments and the verifier checks only a small number of randomly sampled paths from output to input. This yields proving times that are several orders of magnitude faster than SNARK-based approaches (milliseconds instead of minutes) while weakening the soundness guarantees.

A concurrent work, PrivaDE [WTCS26], uses 2PC, ZK proofs, and smart contracts to score dataset utility before purchase. While it shares our cryptographic tools and blockchain setting, it targets data marketplaces rather than inference service accountability.

[3]The Chernoff bound states that if $Y_1, \ldots, Y_N$ are independent Bernoulli random variables with $\mathbb{E}[\sum_i Y_i] = N\mu$, then the probability that their average exceeds μ by more than an additive term ε' decays exponentially in N. Intuitively, the more queries we ask, the harder it is for a weak model to consistently score above its true average by a significant margin.

3 Cryptographic Tools

Notation. We denote by PPT a probabilistic polynomial-time algorithm. $b \leftarrow \mathcal{A}(x)$ means that b is the output of the PPT algorithm $\mathcal{A}$ on input x. We denote with $\mathcal{A}(x;r)$ an execution of the PPT algorithm $\mathcal{A}$ with fixed random coins r. We say $[N] \coloneqq \{1,\ldots,N\}$. We denote by $\lambda \in \mathbb{N}$ the security parameter. All algorithms implicitly take 1^λ as input. A function negl is *negligible*, if for every polynomial p there exists λ_0 such that $\mathsf{negl}(\lambda) < 1/p(\lambda)$ for all $\lambda > \lambda_0$. For a set S, we write $|S|$ for its cardinality. Two families of distributions $\{X_\lambda\}_{\lambda\in\mathbb{N}}$ and $\{Y_\lambda\}_{\lambda\in\mathbb{N}}$ are *computationally indistinguishable*, if for all PPT distinguishers $\mathcal{D}$, $\left|\Pr[\mathcal{D}(X_\lambda)=1] - \Pr[\mathcal{D}(Y_\lambda)=1]\right| \leq \mathsf{negl}(\lambda)$. A Bernoulli random variable $Y \sim \mathrm{Ber}(p)$ takes value 1 with probability p and 0 with probability $1-p$; its expectation is $\mathbb{E}[Y] = p$. NP relations are defined via their associated NP languages. For an NP language $\mathcal{L}$, the corresponding NP relation $\mathcal{R}$ consists of pairs (x,w) such that $|w| \leq p(|x|)$ for some polynomial p and $x \in \mathcal{L}$ if and only if membership can be decided in polynomial time given the witness w.

Definition 1 (Hash Function). *A cryptographic hash function is a deterministic algorithm $H : \{0,1\}^* \to \{0,1\}^\lambda$ mapping inputs of arbitrary length to a fixed-length digest of size λ. We require the following properties:*

- Collision Resistance: *For all PPT adversaries $\mathcal{A}$,* $\Pr\left[\mathcal{A}() \text{ outputs } (x,x') : \begin{array}{c} x \neq x' \\ \wedge H(x) = H(x') \end{array}\right] \leq \mathsf{negl}(\lambda)$.

Definition 2 (Digital Signature Scheme). *A digital signature scheme is a triple of PPT algorithms $\Pi = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{VrfSign})$ where:*

- $\mathsf{KeyGen}() \to (\mathsf{sk}, \mathsf{vk})$*: outputs a secret signing and a public verification keys.*
- $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$*: outputs a signature σ on message m.*
- $\mathsf{VrfSign}(\mathsf{vk}, m, \sigma) \to \{0,1\}$*: outputs 1 if σ is a valid signature on m under vk.*

We require Π to satisfy the following properties:

- Correctness: *For all m,* $\mathsf{VrfSign}(\mathsf{vk}, m, \mathsf{Sign}(\mathsf{sk}, m)) = 1$ *with probability* 1.
- Unforgeability: *For all PPT adversaries $\mathcal{A}$ with access to a signing oracle* $\mathsf{Sign}(\mathsf{sk},\cdot)$, $\Pr\left[\mathcal{A} \text{ outputs } (m^*,\sigma^*) : \begin{array}{c} \mathsf{VrfSign}(\mathsf{vk}, m^*, \sigma^*) = 1 \\ \wedge m^* \notin \mathcal{Q} \end{array}\right] \leq \mathsf{negl}(\lambda)$, *where $\mathcal{Q}$ is the set of messages queried to the signing oracle.*

Definition 3 (Merkle Tree). *A Merkle tree over leaves $\ell_1,\ldots,\ell_N \in \{0,1\}^*$, built using a collision-resistant hash function H, is a complete binary tree where each internal node is H applied to the concatenation of its two children. The root R serves as a succinct commitment to all leaves. It supports:*

- $\mathsf{MerkleCom}(\ell_1,\ldots,\ell_N) \to R$*: outputs the root R.*

- $\mathsf{MerkleOpen}(i, \ell_i) \to \pi_i$*: outputs a proof* π_i *consisting of the sibling hashes along the path from* ℓ_i *to* R*. The size of* π_i *is* $O(\log N)$*.*
- $\mathsf{MerkleVrf}(R, i, \ell_i, \pi_i) \to \{0,1\}$*: outputs* 1 *if* π_i *is a valid opening for* ℓ_i *at position* i *under root* R*.*

We require the following properties:

- Correctness: *For all* i *and all honestly generated proofs* π_i*, it holds that* $\mathsf{MerkleVrf}(R, i, \ell_i, \pi_i) = 1$*.*
- Soundness (Binding): *For any probabilistic polynomial-time adversary* $\mathcal{A}$*, the probability*

$$\Pr\begin{bmatrix} R \leftarrow \mathsf{MerkleCom}(\ell_1, \ldots, \ell_N), \\ (i, \ell_i', \pi_i') \leftarrow \mathcal{A}(R) \\ \mathsf{MerkleVrf}(R, i, \ell_i', \pi_i') = 1 \ \wedge \ \ell_i' \neq \ell_i \end{bmatrix}$$

is negligible in the security parameter.

Definition 4 (Commitment Scheme). *A commitment scheme is a triple of PPT algorithms* $\mathsf{CS} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{Open})$ *where:*

- $\mathsf{Setup}() \to \mathsf{pp}$*: outputs public parameters.*
- $\mathsf{Com}(\mathsf{pp}, m; r) \to c$*: commits to message* m *using randomness* r*.*
- $\mathsf{Open}(\mathsf{pp}, c, m, r) \to \{0,1\}$*: outputs* 1 *if* c *is a valid commitment to* m *under randomness* r*.*

We require the following properties:

- Correctness: *For all* m*,* r*, and* $\mathsf{pp} \leftarrow \mathsf{Setup}()$*, it holds that* $\mathsf{Open}(\mathsf{pp}, \mathsf{Com}(\mathsf{pp}, m; r), m, r) = 1$*.*
- Hiding: *For all PPT adversaries* $\mathcal{A}$*,* $\mathsf{pp} \leftarrow \mathsf{Setup}()$*, and all* m_0, m_1*,*

$$\left|\Pr[\mathcal{A}(\mathsf{Com}(\mathsf{pp}, m_0; r)) = 1] - \Pr[\mathcal{A}(\mathsf{Com}(\mathsf{pp}, m_1; r)) = 1]\right| \leq \mathsf{negl}(\lambda).$$

- Binding: *For all PPT adversaries and* $\mathcal{A}$ *and* $\mathsf{pp} \leftarrow \mathsf{Setup}()$*, it holds that*

$$\Pr\left[\mathcal{A} \text{ outputs } (c, m, r, m', r') : \begin{array}{l} m \neq m' \\ \wedge \mathsf{Open}(\mathsf{pp}, c, m, r) = 1 \\ \wedge \mathsf{Open}(\mathsf{pp}, c, m', r') = 1 \end{array}\right] \leq \mathsf{negl}(\lambda).$$

For simplicity, in the rest of the paper, we omit pp from the notation in the remainder of the paper, as a single commitment scheme is used throughout.

Definition 5 (Two-Party Computation). *Let* $f : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}^* \times \{0,1\}^*$ *be a two-party functionality, where* $f = (f_1, f_2)$ *and* f_i *is the output intended for party* P_i*. A two-party protocol* π *computing* f *between* P_1 *(input* x*) and* P_2 *(input* y*) is an interactive procedure after which each party receives its designated output. Security is defined via the* real/ideal paradigm*.*

In the ***real execution*** $\mathsf{REAL}_{\mathcal{A},\pi}(x, y, \lambda)$*, the parties run* π*, a PPT adversary* $\mathcal{A}$ *may corrupt one party and deviate arbitrarily, while the honest party*

follows π. The output is the joint distribution of the honest party's output and the adversary's view.

*In the **ideal execution** $\mathsf{IDEAL}_{\mathcal{S},f}(x,y,\lambda)$, both parties send their inputs to a trusted functionality $\mathcal{F}$ that computes f honestly and returns f_i to P_i. A PPT simulator $\mathcal{S}$ interacts only with $\mathcal{F}$ and receives only the corrupted party's output. The output is the joint distribution of the honest party's output and the simulator's view. We require:*

- Correctness: *When both parties are honest, each P_i receives $f_i(x,y)$ with probability* 1.
- Security against malicious adversaries: *For every PPT adversary $\mathcal{A}$ corrupting one party, there exists a PPT simulator $\mathcal{S}$ such that $\{\mathsf{IDEAL}_{\mathcal{S},f}(x,\,y,\,\lambda)\}_\lambda \overset{c}{\equiv} \{\mathsf{REAL}_{\mathcal{A},\pi}(x,\,y,\,\lambda)\}_\lambda$.Intuitively, whatever the adversary learns in the real protocol can be simulated from its own input and output alone.*

A protocol π securely realizing a functionality f can be instantiated using general-purpose MPC frameworks; we refer the reader to MP-SPDZ [Kel20] for concrete implementations.

Definition 6 (ZK-SNARK). *A ZK-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) for an NP relation $\mathcal{R}$ is a triple of PPT algorithms* $(\mathsf{Setup},\mathsf{Prove},\mathsf{Verify})$ *where:*

- $\mathsf{Setup}(\mathcal{R}) \to (\mathsf{pk},\mathsf{vk})$*: outputs a proving key and a verification key.*
- $\mathsf{Prove}(\mathsf{pk},\mathbf{x},\mathbf{w}) \to \pi$*: outputs a proof π given statement $\mathbf{x}$ and witness $\mathbf{w}$ with $\mathcal{R}(\mathbf{x},\mathbf{w})=1$.*
- $\mathsf{Verify}(\mathsf{vk},\mathbf{x},\pi) \to \{0,1\}$*: outputs 1 if π is a valid proof for $\mathbf{x}$.*

We require the following properties:

- Completeness: *For all $(\mathbf{x},\mathbf{w})$ such that $\mathcal{R}(\mathbf{x},\mathbf{w})=1$, it holds that $\mathsf{Verify}(\mathsf{vk},\mathbf{x},\mathsf{Prove}(\mathsf{pk},\mathbf{x},\mathbf{w}))=1$.*
- Knowledge Soundness: *For every PPT adversary $\mathcal{A}$ that outputs a valid proof for $\mathbf{x}$, there exists a PPT extractor that recovers a witness $\mathbf{w}$ such that $\mathcal{R}(\mathbf{x},\mathbf{w})=1$, except with probability $\mathsf{negl}(\lambda)$.*
- Zero Knowledge: *There exists a PPT simulator $\mathcal{S}$ such that for all $(\mathbf{x},\mathbf{w})$ such that $\mathcal{R}(\mathbf{x},\mathbf{w})=1$, the simulated and real proofs are computationally indistinguishable.*
- Succinctness: *The proof π has size $\mathsf{poly}(\lambda)$ independent of $|\mathbf{w}|$, and Verify runs in time $\mathsf{poly}(\lambda,|\mathbf{x}|)$.*

Definition 7 (Chernoff Bound). *Let $Y_1,\ldots,Y_n$ be independent Bernoulli random variables with $\mathbb{E}\left[\sum_{j=1}^n Y_j\right]=n\mu$. Then for any $\varepsilon'>0$:*

$$\Pr\left[\frac{1}{n}\sum_{j=1}^{n} Y_j \;\geq\; \mu+\varepsilon'\right] \;\leq\; \exp\left(-\frac{n\varepsilon'^2}{2\mu+\varepsilon'}\right).$$

Definition 8 (Blockchain and Smart Contract). *A blockchain is a distributed, append-only ledger maintained by a set of n nodes running a consensus protocol. It supports the deployment of smart contracts S, which are programs that maintain internal state and process transactions submitted by users. These transactions are publicly verifiable and become immutable once finalized.*

We require the following properties.

- Consistency: *All honest nodes agree on the same sequence of finalized blocks. Formally, for any two honest nodes u and v and any block heights $h_1 \leq h_2$, if u has finalized a block at height h_1 and v has finalized a block at height h_2, then u's chain is a prefix of v's chain. In particular, no PPT adversary controlling fewer than a threshold fraction of nodes can cause two honest nodes to finalize conflicting blocks at the same height.*
- Immutability: *Once a transaction* tx *is finalized at block height h, no PPT adversary can alter, remove, or reorder it.*
- Smart Contract Integrity: *The contract S executes deterministically according to its published code; no party can selectively halt, modify, or deviate from its execution. All state transitions are publicly auditable.*
- Transaction Liveness: *Any transaction* tx *submitted by an honest party is eventually included in a finalized block. Formally, there exists a polynomial bound $T_{\mathsf{confirm}}(\lambda)$ such that* tx *is finalized within T_{confirm} blocks of submission with all but negligible probability.*

4 Model and Assumptions

We describe the parties involved in the protocol, the adversarial scenarios we consider, and the correctness and security goals the protocol is designed to achieve.

4.1 Parties

The protocol involves three types of participants.

Server. The server hosts a proprietary AI model $\mathcal{M}$ and offers inference access through a subscription-based service. It holds a signing key $\mathsf{sk}_{\mathsf{Ser}}$ and registers the corresponding verification key $\mathsf{vk}_{\mathsf{Ser}}$ on-chain. The server's interests are to monetize access to $\mathcal{M}$ without revealing its internal parameters, architecture, or intermediate computations.

Client. The client wishes to subscribe to the server's model and requires assurance that $\mathcal{M}$ meets a performance benchmark B at a correctness threshold τ on a chosen query set Q. It holds a signing key $\mathsf{sk}_{\mathsf{Cli}}$ and registers $\mathsf{vk}_{\mathsf{Cli}}$ on-chain. The client's interests are to verify model quality before committing to payment and to retain recourse if quality degrades afterward.

Smart Contract. A smart contract S deployed on a public blockchain acts as a neutral third party. It stores commitments, holds and transfers deposits, and adjudicates complaints by verifying cryptographic proofs on-chain. It does not interact with the model and learns nothing about B, τ, Q, or any model outputs except what is explicitly submitted to it during a complaint.

4.2 Adversarial Model

We consider fully malicious adversaries: both the server and the client may deviate arbitrarily from the protocol in an attempt to gain an advantage. The smart contract is assumed honest in executing its published code.

Malicious Server. The server may attempt to:

- *Misrepresent model quality during evaluation.* For instance, it may forward the client's benchmark queries to a stronger external model and return its outputs, then switch to a weaker model after payment.
- *Degrade service after subscription.* Once the payment is received, it may deploy a lower-quality model, hoping the client will not detect the change[4].
- *Extract the client's queries or benchmark.* It may attempt to learn Q from the commitment phase, or learn B or τ from its interaction during the pre-purchase evaluation, obtaining an advantage in the model evaluation process.
- *Repudiate answers it returned.* It may deny having produced specific outputs during the subscription phase, in order to avoid liability under a complaint.

Malicious Client. The client may attempt to:

- *Extract model outputs without paying.* It may try to obtain useful answers from the model during the evaluation phase and then decline to subscribe.
- *Bias the evaluation.* It may attempt to construct a query set or challenge sets specifically chosen to make the model fail, in order to obtain a refund it is not entitled to receive back[5].
- *Fabricate a complaint.* It may attempt to submit a fraudulent complaint by forging server responses, switching to an easier-to-fail benchmark, or using a threshold different from the one committed to in Phase 1.

4.3 Protocol Goals

Correctness. When both parties are honest and the model genuinely satisfies the benchmark, the protocol should complete successfully: the client receives the decision bit $b = 1$, and if the payment is transferred to the server no honest complaint is ever performed. Conversely, if the model does not meet the benchmark, the evaluation phase outputs $b = 0$ and no payment is made[6].

Model Privacy. The server's model $\mathcal{M}$ should remain completely hidden from the client throughout the entire protocol. The client should learn nothing about the internal structure of $\mathcal{M}$, including its weights, and architecture, except for what is implied by the received outputs.

[4] For instance, the server may switch to a cheaper, lighter model after payment to reduce operational costs.

[5] Unlike in the previous attack, here the client does subscribe and uses the model legitimately, but subsequently attempts to recover the subscription fee by filing a fraudulent complaint.

[6] We assume no payment is performed since the server will refuse the subscription of the client to avoid an obvious on-chain complaint of the client later.

Output Privacy. The client should learn nothing from the pre-purchase evaluation beyond the single bit b indicating whether the benchmark is met. In particular, no model output should be accessible to the client prior to payment, regardless of how the client deviates from the protocol.

Benchmark Privacy. The server should learn nothing from the pre-purchase evaluation beyond the single bit b. In particular, the benchmark function B and threshold τ should remain hidden from the server throughout the interaction.

Query Binding. The client should not be able to submit, at any point, a query that does not belong to the committed set Q. Any attempt to present a query outside Q as a valid evaluation or complaint input should be rejected.

Non-Repudiation. The server should not be able to deny outputs it returned during the subscription phase. The client should always be able to produce verifiable evidence of the server's responses for use in a complaint.

Subscription Accountability. If after payment the server deploys a model whose quality is strictly below the level agreed upon during the pre-purchase evaluation, the client should be able to detect this degradation with overwhelming probability. Concretely, if the server's model answers queries correctly with average probability strictly below the threshold τ, the probability that the server passes the post-purchase benchmark check decreases exponentially both in N and in the square of the degradation gap $\varepsilon' = \tau - (1-\varepsilon)$. In particular, no polynomially bounded server strategy should allow degradation to go undetected.

5 Protocol Description

We describe the three phases of ASAI in detail. The following public parameters are generated once before Phase 1:

- Commitment-scheme parameters: $\mathsf{pp} \leftarrow \mathsf{Setup}(a)$, pp is stored on S.
- Cli generates the ZK-SNARK parameters for the complaint relation $\mathcal{R}$: $(\mathsf{pk}, \mathsf{vk}_{\mathsf{SNARK}}) \leftarrow \mathsf{Setup}(\mathcal{R})$. Cli sends $\mathsf{vk}_{\mathsf{SNARK}}$ to Ser.
- Cli computes $(\mathsf{sk}_{\mathsf{Cli}}, \mathsf{vk}_{\mathsf{Cli}}) \leftarrow \mathsf{KeyGen}()$ and registers $\mathsf{vk}_{\mathsf{Cli}}$ to S.
- Ser computes $(\mathsf{sk}_{\mathsf{Ser}}, \mathsf{vk}_{\mathsf{Ser}}) \leftarrow \mathsf{KeyGen}()$ and registers $\mathsf{vk}_{\mathsf{Ser}}$ to S.

All subsequent algorithms implicitly use these parameters. For notational simplicity we omit pp and $\mathsf{vk}_{\mathsf{SNARK}}$ from the protocol description.

5.1 Phase 1: Commitment and Challenge Reservation

Client.

(i) Constructs N pairs of queries $(q_1^{(0)}, q_1^{(1)})$, $(q_2^{(0)}, q_2^{(1)})$, $\ldots$, $(q_N^{(0)}, q_N^{(1)})$. For each $i \in [N]$, it creates two leaf values:

- $v_i^{(0)} = q_i^{(0)} \,||\, 0$,
- $v_i^{(1)} = q_i^{(1)} \,||\, 1$.

(ii) Builds a Merkle tree over the $2N$ leaves $v_1^{(0)}, v_1^{(1)}, v_2^{(0)}, v_2^{(1)}, \ldots, v_N^{(0)}, v_N^{(1)}$ and publishes the root $R_Q = \mathsf{MerkleCom}(v_1^{(0)}, v_1^{(1)}, \ldots, v_N^{(0)}, v_N^{(1)})$ to S.

(iii) Selects a benchmark function B with a correctness threshold τ, computes $C_B = \mathsf{Com}(B;\, r_B)$ and $C_\tau = \mathsf{Com}(\tau;\, r_\tau)$, and publishes C_B, C_τ to S.

(iv) Locks a deposit d_{Cli} into S.

Server.

(i) Locks a collateral deposit d_{Ser} into S, which is held as a security bond and subject to slashing upon a successful client complaint.

Challenge Selection via Coin-Flipping. The client and server jointly execute a coin-flipping protocol [Blu81, MNS09, BOO10] to produce a public random bit string $\mathbf{b} = (b_1, \ldots, b_N) \in \{0,1\}^N$.[7] The commitment $C_{\mathbf{b}} = \mathsf{Com}(\mathbf{b};\, r_{\mathbf{b}})$ of $\mathbf{b}$ is recorded on-chain.

For each $i \in [N]$, bit b_i designates which query from pair i is used for pre-purchase evaluation: $q_i^{\text{pre}} \coloneqq q_i^{(b_i)}$, $q_i^{\text{post}} \coloneqq q_i^{(1-b_i)}$. Since $\mathbf{b}$ is derived from randomness contributed by both parties and fixed before any model output is revealed, neither party can bias which query is assigned to which role.

5.2 Phase 2: Private Pre-Purchase Evaluation

(i) **Query Revelation.** For each $i \in [N]$, the client opens the leaf $v_i^{(b_i)}$ in the Merkle tree, revealing $q_i^{(b_i)}$ and the tag b_i, together with the Merkle proof $\pi_i = \mathsf{MerkleOpen}(i, v_i^{(b_i)})$. It sends $\{q_i^{\text{pre}}\}_{i=1}^N$ and $\{\pi_i\}_{i=1}^N$ to the server. The server verifies each opening by checking $\mathsf{MerkleVrf}(R_Q, i, v_i^{(b_i)}, \pi_i) = 1$ and that the tag b_i matches the selected random bit.

(ii) **Server Inference.** The server evaluates its model $\mathcal{M}$ on the revealed queries, obtaining answers $\mathsf{Ans}^{\text{pre}} = \{\mathcal{M}(q_i^{\text{pre}})\}_{i=1}^N$, and submits a signed commitment $\sigma_1 = \mathsf{Sign}(\mathsf{sk}_{\mathsf{Ser}},\, \mathsf{Com}(\mathsf{Ans}^{\text{pre}};\, r_{\mathsf{Ans}}))$ to S.

(iii) **Two-Party Computation.** The client and server execute a 2PC protocol π computing the following functionality f. Both parties have as public input C_B, C_τ, σ_1, the output of $\mathsf{Com}(\mathsf{Ans}^{\text{pre}};\, r_{\mathsf{Ans}})$ and the set of queries $\{q_i^{\text{pre}}\}_{i=1}^N$. The client's private inputs are $(B, r_B, \tau, r_\tau, \mathsf{sk}_{\mathsf{Cli}})$ and the server's private inputs are $(\mathsf{Ans}^{\text{pre}}, r_{\mathsf{Ans}}, \mathsf{sk}_{\mathsf{Ser}})$.

[7] For instance, each party samples a seed, commits to it, and after both commitments are exchanged the seeds are opened and combined as $\mathbf{b} = \mathsf{PRG}(\rho_{\mathsf{Cli}}) \oplus \mathsf{PRG}(\rho_{\mathsf{Ser}})$ where $\mathsf{PRG}(\cdot)$ is a pseudo random generator.

Functionality f

It performs the following steps:

(a) it computes $b_B = \mathsf{Open}(C_B, B, r_B)$. If $b_B = 0$, it aborts;
(b) it computes $b_\tau = \mathsf{Open}(C_\tau, \tau, r_\tau)$. If $b_\tau = 0$, it aborts;
(c) it computes $b_\sigma = \mathsf{Verify}(\mathsf{vk}_{\mathsf{Ser}}, \mathsf{Com}(\mathsf{Ans}^{\mathrm{pre}}; r_{\mathsf{Ans}}), \sigma_1)$. If $b_\sigma = 0$, it aborts;
(d) it computes $b = B\big(\{q_i^{\mathrm{pre}}\}_{i=1}^N, \mathsf{Ans}^{\mathrm{pre}}\big) \geq \tau$;
(e) if $b = 0$, the functionality outputs 0 to both parties;
(f) if $b = 1$, the 2PC additionally computes and outputs the joint attestation $\alpha = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Cli}}}\big(\mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}(b, R_Q, \mathbf{b})\big)$. Both output bit b and attestation α are returned to both parties.

(iv) **On-Chain Settlement.** If $b = 1$, the joint attestation α is submitted to S. The contract verifies both signatures by running $\mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Ser}}, (b, R_Q, \mathbf{b}), \sigma_{\mathsf{inner}}) = 1$ and $\mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Cli}}, \sigma_{\mathsf{inner}}, \alpha) = 1$, where $\sigma_{\mathsf{inner}} = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}(b, R_Q, \mathbf{b})$ is the inner signature extracted from α. If $b = 0$, no attestation is produced. More details in Sec. 5.4.

5.3 Phase 3: Post-Purchase Accountability and Compliance

After payment, if the client suspects that the server has degraded its model, it may invoke the following complaint sub-protocol.

(i) **Challenge Invocation.** For each $i \in [N]$, the client takes $q_i^{\mathrm{post}} = q_i^{(1-b_i)}$ for the queries committed in Phase 1. It sends each query q_i^{post} individually to the server.

(ii) **Signed Responses.** For each query q_i^{post}, the server computes $h_i = H(q_i^{\mathrm{post}})$ and returns the answer $a_i = \mathcal{M}(q_i^{\mathrm{post}})$ together with the signature $\sigma_i = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}(h_i, a_i)$. The client assembles $\mathsf{Ans}^{\mathrm{post}} = \{(q_i^{\mathrm{post}}, a_i, h_i, \sigma_i)\}_{i=1}^N$ locally.

(iii) **ZK-SNARK Complaint Proof.** If the client believes the benchmark is not met, it constructs a ZK-SNARK $\pi_{\mathsf{complaint}}$ for the following relation. Let $C_A = \mathsf{Com}(\{a_i\}_{i=1}^N; r_A)$ be a commitment to the answer set, and recall that $C_B = \mathsf{Com}(B; r_B)$ and $C_\tau = \mathsf{Com}(\tau; r_\tau)$ are the benchmark and threshold

commitments registered in Phase 1. Define the NP relation:

$$\mathcal{R} = \left\{ \begin{array}{l|l} (\mathsf{vk}_{\mathsf{Ser}},\ R_Q,\ C_{\mathbf{b}},\ C_B,\ C_\tau,\ C_A,\ b=0), & \mathsf{Open}(C_B,\ B,\ r_B)=1 \\ & \mathsf{Open}(C_\tau,\ \tau,\ r_\tau)=1 \\ & \mathsf{Open}(C_{\mathbf{b}},\ \mathbf{b},\ r_{\mathbf{b}})=1 \\ & \forall i \in [N]: \\ (B,\ r_B,\ \tau,\ r_\tau,\ \mathbf{b}, r_{\mathbf{b}}, & \quad \mathsf{MerkleVrf}(R_Q,\ i,\ v_i^{(1-b_i)},\ \pi_i)=1 \\ \quad \mathsf{Ans}^{\mathrm{post}},\ r_A) & \quad h_i = H(q_i^{\mathrm{post}}) \\ & \quad \mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Ser}},\ (h_i,a_i),\ \sigma_i)=1 \\ & \quad \mathsf{Open}(C_A,\ \{a_i\},\ r_A)=1 \\ & B\big(\{q_i^{\mathrm{post}}\}_{i=1}^N,\ \{a_i\}_{i=1}^N\big) < \tau \end{array} \right\}.$$

The statement is $\mathbf{x} = (\mathsf{vk}_{\mathsf{Ser}},\ R_Q,\ C_{\mathbf{b}},\ C_B,\ C_\tau,\ C_A,\ b=0)$ and the witness is $\mathbf{w} = (B,\ r_B,\ \tau,\ r_\tau,\ \mathbf{b},\ r_{\mathbf{b}}, \{q_i^{\mathrm{post}}, a_i, h_i, \sigma_i, \pi_i\}_{i=1}^N,\ r_A)$.
The opening checks ensure that the benchmark and threshold evaluated in the circuit are exactly those committed in Phase 1, preventing the client from switching to a different benchmark when filing a complaint. The Merkle verification checks $\mathsf{MerkleVrf}(R_Q, i, v_i^{(1-b_i)}, \pi_i) = 1$ for every $i \in [N]$ bind each post-purchase query to the root R_Q published on-chain in Phase 1, and verify that the tag $1 - b_i$ is consistent with the public $\mathbf{b}$, so no query outside the committed set can appear in a complaint.
The client submits to S: the commitment C_A, the signatures $\{\sigma_i\}_{i=1}^N$, and the proof $\pi_{\mathsf{complaint}}$. The Merkle openings $\{\pi_i\}_{i=1}^N$ are included as part of the ZK-SNARK witness and are therefore not submitted separately to S.

(iv) **On-Chain Arbitration.** The smart contract S verifies the ZK-SNARK $\pi_{\mathsf{complaint}}$ using the statement $\mathbf{x} = (\mathsf{vk}_{\mathsf{Ser}},\ R_Q,\ C_{\mathbf{b}},\ C_B,\ C_\tau,\ C_A,\ b=0)$ constructed from data stored on-chain and the commitment C_A provided by the client. If verification passes, S issues a refund to the client and slashes d_{Ser} accordingly. More details in Sec. 5.4.

5.4 Smart Contract Task

The smart contract S stores the commitments and roots uploaded in Phase 1, receives the signed commitment σ_1 in Phase 2, and verifies the joint attestation α. After successful verification of α, the client decides whether to subscribe by explicitly invoking a *Subscribe* function that transfers d_{Cli} to the server. In Phase 3, the contract verifies a ZK-SNARK complaint and, if valid, issues a refund to the client while slashing the server's collateral deposit.

Smart Contract S

The contract exposes four functions:

1. **Initialize (Phase 1)**: Receives and stores $R_Q, C_B, C_\tau, C_{\mathbf{b}}, \mathsf{vk}_{\mathsf{Cli}}$, $\mathsf{vk}_{\mathsf{Ser}}, \mathsf{vk}_{\mathsf{SNARK}}$ and locks deposits d_{Cli} and d_{Ser}.
2. **Settle (Phase 2)**: On input the joint attestation α, extracts the inner signature σ_{inner} and checks

$\mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Ser}}, (b, R_Q, \mathbf{b}), \sigma_{\mathsf{inner}}) = 1$and $\mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Cli}}, \sigma_{\mathsf{inner}}, \alpha) = 1$.If both verifications succeed and $b = 1$, the contract stores a flag $\mathsf{benchmark\_met} \leftarrow \mathsf{true}$.

3. **Subscribe (client-controlled)**: Callable only by the client. If $\mathsf{benchmark\_met} = \mathsf{true}$, atomically transfers d_{Cli} to the server, finalizing the subscription. If the client never calls this function, d_{Cli} can be withdrawn after a timeout.
4. **Complain (Phase 3)**: On input $(C_A, \{\sigma_i\}_{i=1}^{N}, \pi_{\mathsf{complaint}})$, constructs the statement $\mathbf{x} = (\mathsf{vk}_{\mathsf{Ser}}, R_Q, C_{\mathbf{b}}, C_B, C_\tau, C_A, b = 0)$using the data stored on-chain and the commitment C_A provided by the client, and runs $\mathsf{Verify}(\mathsf{vk}_{\mathsf{SNARK}}, \mathbf{x}, \pi_{\mathsf{complaint}}) = 1$.If the proof is valid, refunds d_{Cli} to the client and slashes d_{Ser}.

6 Security Discussion

We now argue why ASAI achieves the protocol goals identified in Sec. 4.

Correctness: It follows by inspection of the protocol and it holds for the correctness of the 2PC (Def. 5), the correctness of the signature scheme (Def. 2), the correctness of the Merkle Tree (Def. 3), the correctness of the commitment scheme (Def. 4), the smart contract integrity and the transaction liveness of the blockchain (Def. 8).

Model Privacy: The server's model $\mathcal{M}$ remains completely hidden from the client throughout the entire protocol. No part of ASAI ever requires the server to transmit its weights, architecture, circuit structure, or any intermediate activations. In Phase 2 the server evaluates $\mathcal{M}$ locally and feeds only the resulting answers $\mathsf{Ans}^{\mathrm{pre}}$ as a private input into the 2PC; the 2PC security (Def. 5) guarantees that the client obtains nothing except for the decision bit b. In Phase 3, only signed answers to the post-purchase queries are revealed to the client, without disclosing any additional detail about the model beyond what is implied by those answers. The client learns nothing about the internal structure of $\mathcal{M}$ except what is implied by the outputs it is entitled to receive after payment.

Output Privacy: Output privacy follows directly from the security of the two-party computation π realizing functionality f (Def. 5). In the ideal execution the client receives only its designated output b; by the security of the 2PC, the client's view in the real protocol reveals nothing more about $\mathsf{Ans}^{\mathrm{pre}}$. The commitment $\mathsf{Com}(\mathsf{Ans}^{\mathrm{pre}}; r_{\mathsf{Ans}})$ provides no additional leakage because of the hiding property of the commitment scheme (Def. 4). Consequently, no model output is ever accessible to the client prior to payment.

Benchmark Privacy: The server learns nothing about the benchmark function B or the threshold τ beyond the single bit b. The 2PC security (Def. 5) guarantees that the server's output is only b, preventing any leakage of the client's private inputs (B, r_B, τ, r_τ). The commitments C_B and C_τ are hiding (Def. 4), so publishing them on-chain reveals nothing about B or τ. In

Phase 3, the ZK-SNARK reveals nothing about B or τ beyond the fact that a valid complaint was filed, by the zero-knowledge property (Def. 6).

Query Binding: Query binding requires that the client cannot submit, at any point, a query that does not belong to the committed set of pairs. In Phase 1 the client publishes the immutable root R_Q of the Merkle tree over all $2N$ leaves $v_1^{(0)}, v_1^{(1)}, \ldots, v_N^{(0)}, v_N^{(1)}$ on S. In Phase 2, every pre-purchase query opened to the server is accompanied by a Merkle proof; the binding property of the Merkle tree (Def. 3) and the collision resistance of the hash function (Def. 1) used to build the Merkle tree together with blockchain immutability (Def. 8) prevents any PPT adversary from producing a valid opening for a leaf outside the committed set. The same mechanism applies in Phase 3: each post-purchase query q_i^{post} is bound to its leaf $v_i^{(1-b_i)}$ via a Merkle proof checked inside the ZK-SNARK, and the per-leaf bit tag ensures consistency with the public $\mathbf{b}$. Because R_Q is fixed on-chain before any evaluation, the client cannot inject foreign queries at any phase.

Non-Repudiation: Non-repudiation requires that the server cannot later deny outputs it returned during the subscription phase. In Phase 3, for every post-purchase query q_i^{post} the server produces a per-query signature $\sigma_i = \mathsf{Sign}_{\mathsf{sk}_{\mathsf{Ser}}}(H(q_i^{\text{post}}), a_i)$. By the unforgeability of the signature scheme (Def. 2), only the holder of $\mathsf{sk}_{\mathsf{Ser}}$ can generate a valid signature. The verification key $\mathsf{vk}_{\mathsf{Ser}}$ is registered on-chain, so any third party can verify it. The knowledge soundness of the ZK-SNARK (Def. 6) and smart-contract integrity (Def. 8) further guarantee that only genuine server-signed answers can appear in a valid complaint proof.

Subscription Accountability: Since the ZK-SNARK is complete (Def. 6), if $\mathcal{R}(\mathbf{x}, \mathbf{w}) = 1$ the proof is accepting, therefore we argue that it is infeasible for a malicious Ser to cause the generation of a statement and a witness such that $\mathcal{R}(\mathbf{x}, \mathbf{w}) = 0$. It follows from the following three facts.

Pre-committed query split. In Phase 1, before any model output is seen, the client commits to all $2N$ query leaves under root R_Q, and the bit string $\mathbf{b}$ is derived jointly via the coin-flipping protocol and recorded on-chain as $C_{\mathbf{b}} = \mathsf{Com}(\mathbf{b}; r_{\mathbf{b}})$. By the *immutability* property of the blockchain (Def. 8), neither R_Q nor $C_{\mathbf{b}}$ can be altered after Phase 1. Because $\mathbf{b}$ is fixed before any query is evaluated, the server cannot distinguish pre-purchase from post-purchase queries at construction time and therefore cannot selectively behave well only on the queries it expects to be tested in pre-purchase.

Detection probability. Suppose the server deploys a degraded model meaning that it answers each query q_i^{post} correctly with probability at most $1 - \varepsilon_i$, and that the average failure rate $\varepsilon = \frac{1}{N}\sum_{i=1}^{N} \varepsilon_i$ satisfies $1 - \varepsilon < \tau$. Let $\varepsilon' = \tau - (1 - \varepsilon) > 0$ denote the *degradation gap*. Define the *pre-purchase score* as the fraction of correctly answered queries among $\{q_i^{\text{pre}}\}_{i=1}^{N}$, and the *post-purchase score* as the fraction of correctly answered queries among $\{q_i^{\text{post}}\}_{i=1}^{N}$. By construction of the protocol, the post-purchase queries $\{q_i^{\text{post}}\}_{i=1}^{N}$ are independent of the pre-purchase queries $\{q_i^{\text{pre}}\}_{i=1}^{N}$: they are drawn from the complementary leaves of the Merkle tree, selected by the

complementary bits $1 - b_i$. Therefore, the event that the pre-purchase score exceeds τ carries no information about the post-purchase score, and we have $\mathbb{E}[S_{\text{post}}] = \frac{1}{N}\sum_{i=1}^{N}(1-\varepsilon_i) = 1-\varepsilon < \tau$ unconditionally. Let $\mu = 1-\varepsilon$ denote the expected post-purchase score. Let Y_i be the Bernoulli random variable that equals 1 if the server answers q_i^{post} correctly and 0 otherwise. By the Chernoff bound (Def. 7) applied to the N independent random variables $Y_1, \ldots, Y_N$, with $\delta = \varepsilon'/\mu$:

$$\Pr\big[\text{post-purchase score} \geq \tau\big] \leq \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{N\mu} \leq \exp\left(-\frac{N\varepsilon'^2}{2\mu+\varepsilon'}\right), \quad (1)$$

and this bound holds even when conditioning on the pre-purchase score exceeding τ. To make this probability negligibly small, it suffices to choose N large enough relative to ε'. Concretely, for a degradation gap of $\varepsilon' = 0.1$ and $\mu = 0.8$, setting $N = 5000$ query pairs gives an escape probability of at most 2^{-128}. For a larger degradation gap of $\varepsilon' = 0.3$ and $\mu = 0.7$, already $N = 2000$ query pairs suffice to achieve the same guarantee. This confirms that the larger the degradation gap, the fewer queries are needed to detect it. Thus, any server whose model falls short of the threshold by a gap of at least ε' will be caught by the post-purchase complaint mechanism with overwhelming probability.

Complaint soundness. A complaint requires that the ZK-SNARK $\pi_{\mathsf{complaint}}$ is valid. By knowledge soundness (Def. 6), any client that produces a valid proof must possess a witness containing the correct opening of committed benchmark B and $\mathbf{b}$, threshold τ, and genuine server-signed answers to the post-purchase queries. The client therefore cannot fabricate a complaint, switch benchmark or $\mathbf{b}$ values (thanks the binding property of Def. 4), or use a different threshold, since the circuit checks the exact values committed in Phase 1. The binding properties of the commitment scheme and Merkle tree (Definitions 4 and 3) together with the collision resistance of the hash functino (Def 1) and the smart-contract integrity guarantee that no PPT adversary can produce two valid openings to different values. Consistency and transaction liveness of the blockchain ensure that all honest parties observe the same outcome and that slashing is eventually finalized.

7 Experimental Evaluation

We implemented the complaint proof generation protocol described in Sec. 5.3 in the Risc0[8] framework and evaluated its performance on a Dell PowerEdge R760 server running Ubuntu, equipped with two Intel Xeon Platinum 8592+ processors (128 physical cores in total) and 1 TB of system RAM. The implementation is publicly available on GitHub[9].

[8] https://github.com/risc0/risc0

[9] https://anonymous.4open.science/r/secureVerificationModel-25CF

Generating ZK-SNARKs is notoriously resource-intensive, as the prover must arithmetize complex statements and perform expensive cryptographic operations. It is therefore natural to ask whether the complaint mechanism of Sec. 5.3 is practically viable. We answer this question affirmatively by evaluating a concrete implementation of the proof generation pipeline.

Among the cryptographic components of ASAI, ZK-SNARK proof generation is by far the most resource-intensive operation. Merkle tree construction, digital signatures and commitments rely on primitives whose practical efficiency is well-established in the literature. The 2PC component, which can be expensive, evaluates only a lightweight functionality in our protocol: it computes a single benchmark predicate over committed answers. By contrast, the complaint circuit must prove the correct opening of N Merkle membership proofs (one per post-purchase query) which constitutes the dominant cost in SNARK proof generation. We therefore focus our experimental evaluation on the SNARK proof generation of Phase 3, whose practical feasibility is non-obvious.

7.1 Main Technical Choices

The complaint proof $\pi_{\mathsf{complaint}}$ must convince a verifier that the server's responses on the committed challenge queries $\{q_i^{\text{post}}\}_{i=1}^N$ fall below the agreed benchmark threshold τ. A single monolithic ZK-SNARK over all N challenge queries would be impractical for two reasons: (i) the proving time would grow linearly with N, and (ii) the client would have to collect *all* server responses before starting the proof, introducing an unbounded wait. We address both issues by decomposing $\pi_{\mathsf{complaint}}$ into three components that are combined via *recursive proof composition*. A recursive ZK-SNARK system leverages the succinctness and efficient verifiability of ZK-SNARKs so that one proof can verify the correctness of other proofs within its own proving circuit: a single proof π attests that several proofs $\pi_1, \ldots, \pi_k$, each with a corresponding public claim $x_1, \ldots, x_k$, have all been validated correctly. We exploit this paradigm to obtain three components: a *benchmark proof* that certifies the benchmark evaluation against the committed threshold, an *incremental query-verification proof* that accumulates per-query attestations as server responses arrive, and an *aggregation proof* that recursively verifies the previous two and produces a single succinct proof suitable for on-chain verification.

Benchmark proof. The first component proves that the server's responses on the challenge set fail to meet the agreed benchmark threshold τ. Concretely, it verifies the openings of the commitments C_B and C_τ, checks the Merkle inclusion proof $\mathsf{MerkleVrf}(R_Q, i, v_i^{(1-b_i)}, \pi_i) = 1$ $(j \in [N])$, and asserts that the F1-score computed over the collected responses satisfies $\mathsf{F1}(\{q_j^{\text{post}}\}, \{a_j\}) < \tau$. The F1-score is a traditional measure of goodness in supervised learning computed as the harmonic mean of precision (the fraction of predicted positives that are correct) and recall (the fraction of actual positives that are correctly identified) [Pow11]. This component runs once and takes constant time regardless of N.

Incremental query-verification proof. The second component is structured as an Incrementally Verifiable Computation (IVC) [Val08] with one step per query in the challenge set. At step j, the prover extends the running proof to additionally attest: (i) the Merkle membership proof $\mathsf{MerkleVrf}(R_Q, j, q_j)$, showing that query q_j belongs to the pool of N queries committed at the beginning of the protocol, and (ii) the server's digital signature $\mathsf{VrfSign}(\mathsf{vk}_{\mathsf{Ser}}, H(q_j)\|a_j, \sigma_j)$, binding the server to its response a_j on the quer q_j. We used SHA256 to realize the Merkle Tree and the commitment scheme and EdDSA (specifically Ed25519) for digital signatures. At the final step the prover additionally opens the response commitment C_A. Because IVC allows the proof to be updated *incrementally*, the client can generate the proof for step $j-1$ *while waiting* for the server's response to query j, so the proving work is pipelined with the query/response exchanges. The per-step cost is therefore the only latency that the client observes in practice, rather than the total proving time.

Aggregation proof. The third component aggregates the benchmark proof and the final IVC receipt into a single succinct Groth16 [Gro16] proof $\pi_{\mathsf{complaint}}$ via recursive proof composition. The resulting proof can be verified by an on-chain smart contract, satisfying the succinctness requirement.

7.2 Performance Evaluation

Simulation parameters. We evaluated a complaint triggered on 20 queries, simulating a degraded model whose F1-score falls below the committed threshold τ. Table 1 reports the proving time for each proof component.

Table 1: Proving times per component.

Component	**Time (s)**
Benchmark proof	6.75
IVC step (avg. over 20 steps)	21.54
Aggregation proof (Groth16)	26.15

The benchmark proof completes in 6.75 s. The 20 IVC steps exhibit a uniform cost, ranging from 20.97 s to 21.80 s (mean 21.54 s), confirming that the per-step overhead is essentially constant and independent of the accumulated proof state. The final aggregation into a succinct Groth16 proof requires 26.15 s, yielding an end-to-end proving time of approximately 463.70 s ($\approx$ 7.7 minutes) for a challenge of 20 queries.

These results confirm the practical viability of the complaint mechanism. Note that, each IVC step takes roughly 21 s and this is a time period a client typically waits for each server response. In this way, the proving work can be fully pipelined with the subscription interactions, making the *effective* latency perceived by the client negligible. The scheme therefore fits naturally within the accountability process.

Risc0 partitions program execution into fixed-size continuation segments[10], each proven independently. The segment size is the only parameter governing peak RAM usage during proof generation: it can be tuned freely, independently of the complexity of the statement being proved. This decoupling allows arbitrarily large statements to be proved on memory-constrained hardware, and makes peak memory consumption a configurable trade-off rather than an intrinsic property of the protocol. In our implementation, using the default segment size of 2^{22} cycles, peak memory consumption was 8.6 GB, well within the range of commodity hardware

References

ACC+26. Pranay Anchuri, Matteo Campanelli, Paul Cesaretti, Rosario Gennaro, Tushar M. Jois, Hasan S. Kayman, and Tugce Ozdemir. Towards verifiable ai with lightweight cryptographic proofs of inference, 2026. To appear in IEEE SaTML 2026.

APKP24. Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 4316–4330. ACM Press, October 2024. doi:10.1145/3658644.3670316.

Blu81. Manuel Blum. Coin flipping by telephone. In Allen Gersho, editor, *CRYPTO'81*, volume ECE Report 82-04, pages 11–15. U.C. Santa Barbara, Dept. of Elec. and Computer Eng., 1981.

BOO10. Amos Beimel, Eran Omri, and Ilan Orlov. Protocols for multiparty coin toss with dishonest majority. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 538–557. Springer, Berlin, Heidelberg, August 2010. doi:10.1007/978-3-642-14623-7_29.

CSYW24. K. D. Conway, C. So, X. Yu, and K. Wong. opml: Optimistic machine learning on blockchain, 2024. arXiv:2401.17555v2. arXiv:2401.17555.

CWSK24. Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*, EuroSys '24, page 560–574, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3627703.3650088.

Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_11.

Kel20. Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1575–1590. ACM Press, November 2020. doi:10.1145/3372297.3417872.

LXZ21. Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2968–2985. ACM Press, November 2021. doi:10.1145/3460120.3485379.

[10] https://risczero.com/blog/continuations

MNS09. Tal Moran, Moni Naor, and Gil Segev. An optimally fair coin toss. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, March 2009. doi:10.1007/978-3-642-00457-5_1.

Pow11. David Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

PWZ+25. Zhizhi Peng, Taotao Wang, Chonghe Zhao, Guofu Liao, Zibin Lin, Yifeng Liu, Bin Cao, Long Shi, Qing Yang, and Shengli Zhang. A survey of zero-knowledge proof based verifiable machine learning, 2025. URL: https://arxiv.org/abs/2502.18535, arXiv:2502.18535.

QSL+25. Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In *USENIX Security 2025*, pages 2045–2063. USENIX Association, August 2025. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt.

SLZ24. Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 4405–4419. ACM Press, October 2024. doi:10.1145/3658644.3670334.

Val08. Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, March 2008. doi:10.1007/978-3-540-78524-8_1.

WTCS26. Wan Ki Wong, Sahel Torkamani, Michele Ciampi, and Rik Sarkar. PrivaDE: Privacy-preserving data evaluation for blockchain-based data marketplaces. ACM ASIA CCS '26, 2026. doi:10.1145/3779208.3785393.